

KESTREL

Opportunity & Outreach Plan

A focused campaign to turn Kestrel into credible evidence for systems, ML-runtime, compiler, inference, performance and low-level engineering opportunities.

Campaign objective

Use Kestrel to start technical conversations, win interviews, secure paid trials or contract work, earn contributor credibility, and create a realistic path into ML systems and performance-oriented engineering.

Document ID: KST-OUTREACH-001

Version: 1.1

Status: Expanded

Owner: Daniel Ademoye

Date: 7 October 2026

1. Campaign position

Kestrel is not being pitched as a replacement for PyTorch or as a product that these companies should adopt. It is proof of Daniel's ability to reason about and build low-level ML/runtime systems.

Core rule

Daniel is the pitch. Kestrel is the evidence.

What Kestrel should prove

- Daniel can design explicit numerical and tensor semantics instead of hiding behind a high-level framework.
- Daniel can reason about hardware topology, memory spaces, execution backends, correctness and performance.
- Daniel can implement and test a non-trivial C system over multiple months.
- Daniel understands training and inference below the framework layer.
- Daniel can measure performance honestly and explain tradeoffs rather than presenting isolated benchmark numbers.
- Daniel can read technical work by compiler, runtime and inference teams and ask relevant questions.

Primary opportunity types

Type	What to ask for	Best proof
Startup employment	Internship, junior systems/ML-systems role, contract or paid trial	Kestrel release + technical write-up
Open-source route	Issue, bounty, PR, review, contributor relationship	Merged contribution + technical discussion
Big-tech networking	Technical conversation, direction, future role awareness, later referral	Specific question tied to public work
Formal early-career role	Interview consideration	Resume + Kestrel + formal application
Founder/CTO introduction	Short conversation about fit and what could be useful	Concise project evidence; no generic biography

2. Release evidence package

The outreach campaign becomes materially stronger when a reviewer can verify Kestrel quickly. The repository should answer the following without a call with Daniel.

Asset	Required content
README	One-sentence definition; build/run instructions; supported backends; example training flow; current limitations.
Architecture	One diagram showing NN API -> autodiff -> tensor/numerical layer -> heterogeneous runtime.
Correctness tests	Dtype semantics, topology/registry behavior, tensor/view rules, backend correctness, gradient checks, optimizer behavior.
MNIST proof	Canonical 784 -> 256 -> 10 training result; >=97% CPU test accuracy target.
Backend matrix	Scalar, AVX2, Vulkan and CUDA status; clearly mark unavailable or incomplete targets.
Benchmarks	Kernel, transfer, synchronization and end-to-end results separated; machine and compiler metadata included.
Technical write-up	What Daniel designed, what changed during development, one hard bug/decision, one performance finding, limitations.
Release tag	v2.0 or agreed release identifier, release notes, reproducible commands.
Resume entry	3-4 lines focused on systems work, not feature count.

Repository test

A strong systems engineer should be able to spend five minutes in the repository and understand what Kestrel is, what Daniel built, what works, how it was tested, and what is still missing.

3. Campaign sequence

Outreach begins before the final release, but the ask changes as Kestrel matures.

Wave	Period	Purpose	Ask
A - Seed relationships	Oct 4 - Nov 2	Identify relevant founders, maintainers and engineers. Start a small number of technical conversations.	Feedback or one specific technical question. No mass job request.
B - Show progress	Nov 3 - Dec 14	Share concrete progress with people who responded; begin selective formal applications and contribution attempts.	Review a benchmark/design note; point to a role, issue or person if relevant.

Wave	Period	Purpose	Ask
C - Release campaign	Dec 15 - Jan 18	Use the finished evidence package for direct startup outreach and formal early-career applications.	Internship, junior role, contract, paid trial, interview or referral when appropriate.
D - Follow-through	Jan 19 onward	Convert conversations into work; continue OSS proof and role applications.	Follow-up with new technical evidence, not “just checking in.”

4. Targeting rules

- Target teams that can understand Kestrel: compilers, runtimes, inference, kernels, accelerators, hardware enablement, performance, systems software, HPC and low-level ML infrastructure.
- Prefer people with a public technical artifact Daniel has actually read: engineering post, paper, talk, repository, issue or benchmark.
- Do not send the same message to founders, staff engineers, maintainers and recruiters.
- Do not ask a big-tech engineer for a job in the first message. Start from their work.
- Do not cold-apply to a project whose contribution path explicitly requires contribution first.
- Do not treat “remote” as “hire from any country.” Check location and work-authorization requirements before spending application effort.
- Do not send 500 generic messages. A smaller number of technically specific messages is the campaign.

Minimum personalization standard

Every message must contain one sentence that could not be sent unchanged to another company or engineer.

5. Opportunity map

Verified against official public material on 4 October 2026. Open roles and eligibility can change; re-check before applying.

Priority	Target	Why it fits Kestrel	Primary route
S	tinygrad / tiny corp	Small NN framework; scheduler, lowering and execution engine; hiring explicitly uses contributions/bounties as a filter.	Contribute or complete bounty first; then employment conversation.
S	yasp	Agentic ML compiler; hardware-specific kernel optimization; general application currently available.	General application + precise founder/engineer message.
S	GPU.ai	GPU infrastructure; open application; explicitly welcomes interns.	Open application + targeted note.
S	General Compute	Inference on alternative silicon; model bring-up/compiler work; founders invite direct email when role does not fit.	Direct founder/careers email after Kestrel has visible compiler/runtime evidence.
S	NVIDIA early talent	Current 2027 systems software internship includes runtimes, compilers, Linux, low-level programming, XLA/TVM/Halide.	Formal application first; technical networking in parallel if eligible.
A	Modular	Compiler, runtime, kernel, inference optimization and hardware-enablement roles.	Apply where geography/level fits; network with compiler/runtime engineers.
A	Tenstorrent	Low-level kernels, AI compiler, systems software and university/recent-grad programs.	University/recent-grad route + engineer networking.
A	Cerebras	Runtime, kernel, GPU inference and ML systems performance roles.	Formal role if eligible; engineer outreach after release.
A	Baseten	Inference runtime, custom kernels and topology-aware performance work.	Engineer networking; selective applications as Kestrel evidence strengthens.
A	Together AI	Research across inference and kernels; full-stack performance work.	Research/engineer outreach tied to a specific article or kernel project.
A	Gimlet Labs	Multi-silicon AI infrastructure spanning compilers, systems and performance; invites research contact.	Technical email; treat as stretch.
A	Onyina	Remote-first Africa/UK/US; low-latency inference work; general application currently open.	General application + concise technical note.
B	Meta	Heterogeneous AI hardware and kernel optimization; KernelEvolve is directly relevant to automated low-level optimization.	Technical DM/email to relevant authors/engineers; no first-message referral ask.
B	OpenXLA / Google ecosystem	Open-source compiler targeting GPUs, CPUs and ML accelerators; accepts external contributions.	Contribution/community route + targeted Google/OpenXLA networking.

Priority	Target	Why it fits Kestrel	Primary route
B	OpenAI / Triton ecosystem	Inference, compilers, runtimes and performance roles; Triton is a public contribution route.	OSS contribution + technical networking; current direct roles mostly senior.
B	Bloomberg	Large C++/systems engineering footprint and formal early-career route.	Formal early-career application + C++/performance engineer networking.

6. Priority target notes

tinygrad / tiny corp

Fit. Closest project-level comparison to Kestrel in the list: tensors, scheduling, lowering, accelerators and neural-network execution.

Current public signal. tinygrad states that it is hiring software engineers, that talented interns are acceptable, that bounties are used to judge fit, and that software applicants who have not contributed will not be considered. [S1]

Daniel action. Do not lead with a cold job email. Read the developer docs, choose a bounded issue or bounty, contribute, then use the contribution as the employment conversation.

yasp

Fit. Strong match for Kestrel's eventual hardware-aware execution story: compiler, kernel optimization and multiple hardware targets.

Current public signal. yasp currently describes an agentic ML compiler, lists compiler/kernel experience in its roles, and has a general application asking for links to things applicants have built. [S2]

Daniel action. Apply through the general route after Kestrel has credible benchmarks. Send a separate short note referencing one specific technical claim from yasp's compiler work.

GPU.ai

Fit. Good practical target because it sits around GPU infrastructure rather than requiring frontier research credentials.

Current public signal. GPU.ai currently has an open application and says it is always looking for interns. [S3]

Daniel action. Send a concise general application once Kestrel demonstrates actual GPU work; emphasize systems execution and measurement, not “AI enthusiasm.”

General Compute

Fit. High technical match: alternative silicon, model bring-up, ML compilers, inference correctness and runtime integration.

Current public signal. General Compute currently lists model bring-up / ML compiler work and explicitly says candidates who do not see their role can email the founders/careers address. [S4][S5]

Daniel action. Treat as a stretch. Contact only after Kestrel has a clean technical story around backend correctness, numerics and heterogeneous execution.

NVIDIA

Fit. The clearest formal early-career target in this map.

Current public signal. NVIDIA's current 2027 Systems Software Engineering internship describes GPU system software, runtimes, math libraries, compilers, Linux, low-level programming, computer architecture and tools such as XLA, TVM and Halide. [S6]

Daniel action. Apply formally if eligibility/work authorization permits. Then contact engineers only when Daniel has read their public work and can ask a real question.

7. Open-source credibility track

Open-source work is not a side activity. For several targets it is the highest-signal way to prove that Daniel can enter an unfamiliar production codebase, understand review standards and ship a bounded change.

Project	What it demonstrates	Entry strategy
tinygrad	Neural-network execution, scheduling, lowering and accelerator thinking.	Issue/bounty -> PR -> maintainer discussion. Required before tiny corp software hiring.
OpenXLA	ML compiler, optimization passes, hardware-specific lowering and benchmark discipline.	Read contribution guide -> build locally -> documentation/test/small issue -> performance work later.
MLIR	Compiler infrastructure and large C++ project contribution.	Start with docs/tutorial/bug or well-scoped code change.
Apache TVM	ML compiler stack, C++/Python engineering, tests and review process.	Pick scoped PR; follow lint/test/review process.
vLLM	Production inference stack, Python/C++/CUDA surface and large contributor community.	Use selected good-first issues; do not start with a giant performance rewrite.
Triton ecosystem	GPU kernel/compiler work relevant to performance engineering.	Learn language/compiler through a small issue or extension; pair with Kestrel GPU backend work.

Contribution standard

One understood, reviewed and merged contribution is worth more for this campaign than ten AI-generated pull requests. OpenXLA explicitly requires authors to understand their code and warns against long-winded AI-generated PR material. [S15]

8. Big-tech engineer networking

The first message is a technical conversation, not a disguised referral request.

Target	People to identify	Conversation anchor
Meta	KernelEvolve, PyTorch compiler, Triton, MTIA, performance/inference engineers	A specific kernel-search, correctness, profiling or heterogeneous-hardware question tied to Meta public work.
NVIDIA	CUDA libraries, TensorRT, compiler, runtime, GPU system software, performance engineers	A concrete Kestrel GPU/runtime issue or question tied to an engineer's talk/post/project.
Google / OpenXLA	XLA, JAX compiler, TPU runtime, MLIR contributors	A compiler optimization, fusion, backend or contribution question.
OpenAI	Triton, model performance, inference systems, runtime/compiler engineers	A Triton or runtime-performance question. Do not approach senior-role postings as if they are junior openings.
Bloomberg	C++, low-latency/data systems, performance and infrastructure engineers	C/C++ performance, systems design or measurement question; use formal early-career channel for applications.

Meta example. KernelEvolve publicly describes optimizing kernels across NVIDIA GPUs, AMD GPUs, Meta MTIA and CPUs using correctness checks, profiling and search. That is a legitimate technical bridge from Kestrel to the people who worked on it. [S12]

OpenAI example. Current GPT infrastructure roles explicitly mention compilers, runtimes, kernel optimization, performance engineering, accelerator architecture, Triton, CUDA and MLIR. These are evidence of field fit, not an indication that the current roles are junior. [S13][S14]

9. Message strategy by recipient

Startup founder / CTO

Purpose: get a serious look at Daniel for an internship, junior role, contract or paid trial.

- 1 sentence showing Daniel knows what the company builds.
- 1 sentence defining Kestrel.
- 2-3 strongest pieces of evidence.
- Direct ask: internship, junior role, contract or short paid trial.
- Repository / write-up link.
- Keep the entire message short enough to read on a phone.

Engineer at Meta, NVIDIA, Google, OpenAI, Bloomberg or a larger startup

Purpose: start a technical relationship.

- Reference their actual public work.
- Explain the Kestrel subsystem Daniel is currently working on.
- Ask one specific question.
- Link the repo only if it helps answer the question.
- Do not ask for a referral in message one.

Open-source maintainer

Purpose: contribute useful work.

- Read contribution guidance before contacting anyone.
- Use the public issue/RFC/PR process unless maintainers explicitly invite direct contact.
- Ask questions that show Daniel already investigated the problem.
- Never submit code Daniel cannot explain line by line.

Recruiter / formal role

Purpose: get the application evaluated against a specific opening.

- Name the exact requisition.
- Lead with fit to its stated work.
- Use Kestrel as evidence for the required systems skills.
- Do not send a general technical essay.

10. Message structures

Founder structure

Subject: C/ML-systems engineer - Kestrel runtime project

Hi [Name] - I've been following [specific company work]. I'm building Kestrel, a neural-network library/runtime in C, to learn ML systems from the runtime up: tensor/numerical semantics, autodiff, CPU/GPU backends and hardware-aware execution. [One specific reason the company overlaps.] I'm looking for junior, internship, contract or paid-trial work in this area. Repo/write-up: [link]. Would you be open to taking a look?

Engineer structure

Hi [Name] - I read your work on [specific project/post]. I'm building a C neural-network runtime called Kestrel and I'm currently working through [specific subsystem]. Your point about [technical detail] made me rethink [design/measurement issue]. One thing I'm trying to understand: [specific question]. Repo/write-up if useful: [link].

Follow-up structure

Do not send “just checking in.” Follow up only with new information: a merged PR, release, benchmark result, design revision, or a concise answer to a question they previously raised.

11. Outreach tracker

Use one spreadsheet or CSV. Every contact gets one row. No message should be sent without a recorded reason for targeting that person.

Field	Meaning
Target ID	Stable internal ID.
Organization	Company/project.
Person	Name and public role.
Category	Founder / engineer / maintainer / recruiter.
Why this person	Specific technical or hiring relevance.
Public work	Post, repo, paper, talk or role that anchors the message.
Channel	Email / LinkedIn / X / GitHub / application portal.
Wave	A / B / C / D.

Field	Meaning
Message date	When first contact was sent.
Status	Draft / sent / replied / conversation / applied / interview / closed.
Next action	Single concrete next step.
Follow-up date	Only if a meaningful follow-up is justified.
Outcome	Referral, application, PR, call, rejection, no response, paid work, etc.

12. Campaign operating targets

These are internal process targets, not predicted response rates.

By end of Wave C	Target
High-fit organizations researched	30-40
Individual technical contacts researched	40-60
Tailored founder/engineer messages sent	30-50
Formal applications submitted	Only roles that actually fit eligibility/level; target 8-15
Open-source contribution routes attempted	2-3
Useful technical conversations	5+
Concrete outcomes	At least one of: interview loop, paid trial/contract, bounty payment, referral, or sustained maintainer relationship

Quality control

If volume rises by lowering personalization or sending messages Daniel cannot defend technically, reduce volume.

13. Weekly campaign cadence

Day	Action
Monday	Research 3-5 new targets; update tracker; choose one public technical artifact from each high-priority target.
Tuesday	Draft and send 2-4 highly targeted messages.

Day	Action
Wednesday	Kestrel build work; capture one technical artifact worth sharing later.
Thursday	Open-source issue/contribution work or formal applications.
Friday	Send 2-4 additional targeted messages; respond to active conversations.
Weekend	Review campaign data. Remove weak targets. Prepare only justified follow-ups. Update public Kestrel evidence.

14. What not to do

- Do not pitch Kestrel as production competition to mature frameworks.
- Do not send the same paragraph to 100 people.
- Do not lead with financial hardship. Lead with work and fit.
- Do not exaggerate backend completion, GPU support, benchmarks or training results.
- Do not describe planned features as implemented features.
- Do not ask engineers for referrals before establishing relevance.
- Do not submit AI-generated open-source code that Daniel cannot explain.
- Do not chase every AI company. Prefer teams that can understand the engineering signal Kestrel provides.
- Do not let outreach consume the time required to finish Kestrel.

15. Pre-send checklist

Check	Pass condition
Target fit	Their work clearly overlaps systems/runtime/compiler/inference/performance or a formal early-career path.
Specificity	Message names a real project, post, issue, role or technical problem.
Truth	Every Kestrel claim is implemented and public, or explicitly described as in progress.
Ask	One clear ask.
Length	Readable without scrolling through an essay.
Link quality	Repo/write-up opens and is understandable without private context.
Eligibility	Location/work authorization/student status checked before formal application.
Tracker	Contact row exists before send.

16. What counts as success

The campaign is successful before it produces a full-time offer if it creates credible technical signal and access that Daniel did not have before.

- A founder asks Daniel to interview or complete a paid technical task.
- A startup creates or considers a junior/intern/contract path because Kestrel is relevant.
- A major open-source project merges Daniel's contribution and maintainers know his work.
- An engineer at a target company gives substantive technical feedback and keeps the conversation open.
- A relevant engineer refers Daniel after seeing the work.
- Daniel enters a formal interview loop for systems, compiler, inference, runtime or performance work.
- A bounty, contract or paid trial produces immediate income while building stronger evidence.

17. Verified source register

Official/public sources checked on 4 October 2026. Links are included so the target list can be re-verified before outreach.

- [S1] tinygrad / tiny corp - hiring and contribution route - <https://tinygrad.org/>
- [S2] yasp - careers, compiler roles and general application - <https://yasp.ai/company/careers>
- [S3] GPU.ai - careers, open application and internships - <https://gpu.ai/careers>
- [S4] General Compute - careers and direct careers contact - <https://www.generalcompute.com/careers>
- [S5] General Compute - model bring-up / ML compiler role - <https://www.generalcompute.com/careers/model-bringup-compiler-engineer>
- [S6] NVIDIA - 2027 Systems Software Engineering internships - <https://jobs.nvidia.com/careers/job/893397026201-nvidia-2027-internships-systems-software-engineering-us-ca-santa-clara?domain=nvidia.com>
- [S7] Modular - current compiler/runtime/inference careers - <https://www.modular.com/company/careers>
- [S8] Tenstorrent - careers / software / systems - <https://tenstorrent.com/careers>
- [S9] Cerebras - current open positions - <https://www.cerebras.ai/open-positions>
- [S10] Baseten - careers - <https://www.baseten.co/resources/careers/?department=engineering>
- [S11] Together AI - research blog / kernels / inference - <https://www.together.ai/research-blog>
- [S12] Meta - KernelEvolve engineering post - <https://engineering.fb.com/2026/04/02/developer-tools/kernelevolve-how-metas-ranking-engineer-agent-optimizes-ai-infrastructure/>
- [S13] OpenAI - Systems Generalist, GPT Infrastructure - <https://openai.com/careers/systems-generalist-gpt-infrastructure-san-francisco/>
- [S14] OpenAI - Software Engineer, GPT Infrastructure - <https://openai.com/careers/software-engineer-gpt-infrastructure-san-francisco/>
- [S15] OpenXLA - contribution guide - <https://openxla.org/xla/contributing>
- [S16] OpenXLA - project overview - <https://openxla.org/xla>
- [S17] MLIR - contributing - https://mlir.llvm.org/getting_started/Contributing/
- [S18] Apache TVM - contributor guide - <https://tvm.apache.org/docs/contribute/>
- [S19] vLLM - contribution entry points - <https://github.com/vllm-project/vllm/contribute>
- [S20] Bloomberg - early careers - <https://bloomberg.com/company/early-career/>
- [S21] Bloomberg - technology overview - <https://www.bloomberg.com/company/values/tech-at-bloomberg/>
- [S22] Gimlet Labs - join/research contact - https://gimletlabs.ai/join_us
- [S23] Onyina - careers/general application - <https://onyina.com/careers/>

Re-verification rule

Before every formal application or direct employment pitch, open the official careers/project page again. Roles, eligibility and hiring routes change faster than this document.

Appendix A - Technical creator & streamer relationship track

This appendix adds a relationship-building route for programming creators, streamers and independent systems engineers. It complements the existing founder, engineer, maintainer and formal-application routes; it does not replace them.

Core rule

Relationship first. Kestrel later.

The objective is to become a familiar, technically serious participant in communities that overlap Kestrel - not to appear once, drop a repository link and ask for attention.

A.1 Why this track exists

- Programming creators already spend public time discussing code, architecture, tools and projects. That creates legitimate surfaces for technical interaction that do not require an employment pitch.
- Kestrel can eventually provide a concrete reason to continue the conversation: runtime design, C systems work, autodiff, tensor semantics, memory, SIMD, GPU backends, topology and performance measurement.
- Daniel Builds strengthens this route by giving contacts a quick way to see Daniel explain technical ideas and document project decisions, rather than evaluating the repository cold.

The track is not based on an assumption that any individual will reply. No response is normal. The success condition is a small number of substantive technical relationships, not universal acknowledgement.

A.2 How it plugs into the existing campaign waves

Wave	Creator relationship action	What not to do
A - Seed relationships Oct 4 - Nov 2	Follow their current work; watch streams when useful; join public communities where invited; make occasional genuine technical comments or questions. Record the public artifact that makes each person relevant.	Do not cold-DM a finished biography, ask for a referral, repeatedly tag them, or paste Kestrel into unrelated chat.
B - Show progress Nov 3 - Dec 14	If a real interaction already exists, share one relevant Kestrel design note, benchmark, bug, build-log video or technical question when it naturally connects to their work.	Do not manufacture an excuse to send every milestone to every person.
C - Release campaign Dec 15 - Jan 18	Send Kestrel selectively to the strongest-fit contacts. Lead with one technical hook and one question. Link a 2-minute demo/write-up before asking anyone to inspect the full repository.	Do not make the first serious DM a job/referral request. Ask for critique or perspective when that is the genuine purpose.
D - Follow-through Jan 19 onward	Continue only when there is new evidence: a benchmark, release, merged contribution, design revision, answer to an earlier question, or a conversation they explicitly kept open.	Never send “just checking in” repeatedly. Silence is not a queue that must be cleared.

Community standard

Stream chat visibility is not the same as a relationship. Familiarity becomes useful only when it is attached to repeated, relevant, respectful technical interaction. Do not force private contact when a creator clearly prefers public community channels.

A.3 Creator / streamer target map

These are relationship targets because their public work overlaps low-level programming, systems, languages, tools, performance or the broader culture of building things to understand them. Priority here means topical fit and route clarity - not predicted reply probability.

Target	Why Kestrel fits	Primary relationship route	Possible Kestrel hook
Tsoding	Recreational programming; extensive C/C++ work, compiler/VM projects, arena allocator and from-scratch builds. [S24]	Community/stream first; release DM only after genuine familiarity.	Kestrel topology, arenas, VM/runtime design, C build systems.
Bashbunni	Programming streamer with public Twitch/YouTube/social routes. [S25]	Participate in public streams/community; use a direct message only when there is a specific technical reason.	Building in public, systems learning, Kestrel engineering journey.
Janet A. Carr (@janetacarr)	Independent developer/consultant; streams software development and writes about functional programming and software design. [S26]	Stream/community first; later share one design problem rather than a generic project pitch.	Computation graphs, programming-language ideas, architecture tradeoffs.
Karl Zylinski	Programming educator and game developer focused on Odin and demystifying low-level programming. [S27]	Public writing/video discussion or concise email after a relevant Kestrel milestone.	Manual memory, low-level API design, performance, learning by building.
Ginger Bill	Creator of Odin; systems-language and low-level software work with strong emphasis on performance and craftsmanship. [S28]	Odin/Handmade community route first; direct contact only with a concrete design question.	C runtime design, memory, data-oriented choices, backend abstractions.
Jacob Sorber	Computer-science professor and educator with extensive systems/C teaching and embedded-systems work. [S29]	Email is legitimate, but keep it technical and bounded.	C ownership/failure contracts, memory, processes/threads, systems pedagogy.
Jon Gjengset	Principal engineer, Rust educator and open-source contributor with systems/distributed-systems background. [S30]	Public writing/Discord/community first; concise message around one hard systems question.	API invariants, ownership, performance measurement, large-system design.

A.3 Creator / streamer target map - continued

Target	Why Kestrel fits	Primary relationship route	Possible Kestrel hook
Amos Wenger / fasterthanlime	Independent open-source engineer, teacher and video maker known for long-form systems explorations. [S31]	Read a relevant article first; public/community interaction before direct outreach.	Debugging a hard systems problem, memory/performance investigation, technical writing.
jdj	Programming creator with public C examples and from-scratch project content. [S32]	Public video/social/community interaction; later concise project message.	C runtime implementation, graphics/build-from-scratch culture, Kestrel demo.
Ryan Fleury	Low-level C/tools work and Handmade Network community; works on performance-critical developer tools. [S33]	Handmade community route is preferable to a generic cold DM.	Custom allocators, parsers, tooling, runtime representation, performance.
The Chernob	C++/engine creator with an explicit public code-review submission route. [S34]	Use the code-review route if Kestrel becomes a suitable review candidate; otherwise community/public channels.	Ownership, API design, memory, scalability and runtime architecture.
Andreas Kling	Created SerenityOS and now builds Ladybird; long-running public systems-development work. [S35]	Community/public project route; direct message is release-stage and highly concise.	Large systems built from scratch, C/C++ architecture, runtime/tooling design.
Mitchell Hashimoto	Built major infrastructure tools; now works on Ghostty/libghostty and publishes detailed systems/project writing. [S36]	Read a specific technical post first; release-stage outreach should reference it directly.	SIMD, memory leaks, terminal/runtime performance, building large technical projects.

A.4 Relationship operating rules

- Do not optimize for reply rate. Optimize for whether the message gives the recipient a technically legitimate reason to care.
- Do not behave like a fan trying to become visible. Participate because the technical discussion is useful; familiarity is a side effect.
- One person, one reason. Each target needs a public work anchor and a Kestrel hook that would make no sense if copied to somebody else.
- Keep public channels public. If a creator primarily interacts through stream chat, Discord, issues or community spaces, do not force a private route.
- Release-stage contact is selective. Send Kestrel only to the subset whose work genuinely overlaps what Kestrel became.

A.5 Message patterns and evidence

The creator/streamer message is different from the founder and recruiter messages in the main plan. Its first purpose is technical conversation.

Relationship-stage message

“Hey [Name] - I was watching/reading your work on [specific thing]. I ran into a related problem while building Kestrel, a neural-network runtime in C: [one-sentence technical problem]. I ended up [short design decision]. I was curious how you think about [specific question].”

Release-stage message

“Hey [Name] - I finally got Kestrel to a point where I am comfortable showing it. It is a C neural-network runtime with [only implemented highlights]. The part I think might be up your alley is [one hook tied to their work]. Short demo/write-up: [link]. If you ever take a look, I would especially value your take on [one question].”

Evidence package for this track

- Repository: the same five-minute repository test defined in Section 2 still applies.
- Short technical demo: a reviewer should be able to understand the interesting subsystem before deciding whether to inspect source.
- Daniel Builds: twice-monthly technical content can support this campaign - one deep concept/code-along video and one project/build-log style video. Kestrel videos should show decisions, mistakes, benchmarks and unresolved tradeoffs rather than function as advertisements.
- Technical write-up: one hard design change is more useful than a long feature inventory. The topology redesign is a strong example: flat device model -> physical devices + execution targets + backend instances + memory spaces after studying MIG, Level Zero, SR-IOV and device groups.

A.6 Tracker additions

For creator/streamer contacts, add these fields to the existing outreach tracker: community joined, public interactions, last technical artifact consumed, private contact invited?, and Kestrel hook. A direct message should not be sent merely because a target row exists.

A.7 Appendix source register

- [S24] Tsoding - official GitHub / project profile - <https://github.com/tsoding>
- [S25] Bashbunni - official link hub - <https://linktr.ee/bashbunni>
- [S26] Janet A. Carr - GitHub/profile and public development work - <https://github.com/janetacarr>
- [S27] Karl Zylinski - programming educator / low-level programming - <https://zylinski.se/>
- [S28] Ginger Bill - Odin creator / systems programming - <https://gingerbill.org/>
- [S29] Jacob Sorber - Clemson profile / systems and embedded computing - <https://people.computing.clemson.edu/~jsorber/>
- [S30] Jon Gjengset - official site - <https://thesquareplanet.com/>
- [S31] Amos Wenger / fasterthanlime - official site - <https://fasterthanli.me/>
- [S32] jdh - public profile links - <https://jdh.gg/>
- [S33] Ryan Fleury - Handmade Network profile - <https://handmade.network/m/ryanfleury>
- [S34] The Chernob - public Code Review submission route; verify current instructions before sending - <https://www.youtube.com/@TheCherno>
- [S35] Andreas Kling - official site / Ladybird work - <https://awesomekling.com/>
- [S36] Mitchell Hashimoto - official site / Ghostty and systems writing - <https://mitchellh.com/>

Re-verification rule

Before direct outreach, re-open the target's current public profile/community page. Contact routes, active projects and preferred interaction channels change. Do not rely on this appendix as permanent contact metadata.